WEEK 2 · LECTURE 2

sizeof, Overflow, and 0.1 + 0.2

Why Values Betray Naive Arithmetic

THE MYSTERY, REVISITED

0.1 + 0.2 == 0.3 printed NOT equal

Last lecture ended with a mystery. Today: the real answer — plus one other bug that will absolutely bite you this semester if you don't see it here first, in a safe room.

- Comparison, logical & ternary operators — and short-circuit evaluation
- Integer overflow, in depth — and a bug that never crashes
- Floating point — and the answer, finally
- Char/string/bool literals, text encoding, and implicit/explicit conversion

COMPARISON OPERATORS

Every comparison produces a bool

```
int a = 7, b = 2;
std::cout << (a == b);   // 0
std::cout << (a != b);   // 1
std::cout << (a > b);    // 1
std::cout << (a <= b);   // 0
```

Key idea std::cout prints a bool as 1 or 0, not the words true/false. == compares — a completely different operator from = (assigns); mixing them up is one of the most common beginner bugs, and week 3 shows how it slips past the compiler.

LOGICAL AND — A REAL EXAMPLE

&& combines two bool expressions into one

```
int age = 20;
bool isAdult = (age >= 18);
bool hasID   = true;
std::cout << (isAdult && hasID);   // 1 -- both are true
```

Key idea && (AND) and || (OR) combine two bool expressions into one; ! (NOT) flips a single one. Next question: does the right side of && or || always actually run?

SIDE EFFECTS, STEP BY STEP

Predict each line before running it

```
int a = 1, b = 2, c = 0;
std::cout << a++ - 1;          // 0 -- old value 1, then a becomes 2
std::cout << (a && b || !c);   // 1 -- a is 2 now, so a && b is true
std::cout << b / ++a;          // 0 -- ++a makes a 3; 2 / 3 truncates
```

Key idea Each line's side effect carries into the next. Now the real question: when the LEFT side of && or || has already decided the answer, does the compiler still bother evaluating the right side?

SHORT-CIRCUIT: || STOPS EARLY

The right side isn't always evaluated

```
int x = ++a || (++b && ++c);   // a is 3 here -- ++a makes it 4, already true
std::cout << a << b << c << x;   // 4201 -- ++b, ++c NEVER RUN
```

Key idea || stops the instant its left side is true. The skipped right side is NEVER evaluated — side effects and all. This is a language guarantee, not an optimization.

SHORT-CIRCUIT: && STOPS EARLY

Same guarantee, the other operator

```
a = b = c = -1;
int y = ++a && ++b && ++c;   // ++a takes a to 0 -- FALSE
std::cout << a << b << c << y;   // 0-1-10 -- ++b, ++c NEVER RUN
```

Why this matters This is exactly the pattern behind a guard condition — denom != 0 && num / denom > threshold. The zero-check on the left protects the division on the right, and it only works because the right side never runs once the left is already false.

THE TERNARY OPERATOR

condition ? a : b

```
bool isMember = true;
double fee = isMember ? 5.0 : 10.0;
std::cout << fee;   // 5
```

Discuss Next week's if/else can write this same choice as a full statement. Which form do you think reads more clearly here?

PRECEDENCE, CONTINUED

Four more operators join the ladder

```
std::cout << (3 + 2 > 4 && 1 < 2);   // 1
// reads as: ((3 + 2) > 4) && (1 < 2)
```

Key idea High to low: arithmetic, then comparison, then &&, then ||, then ?:, then assignment, then the comma operator. && binds tighter than || — parenthesize anyway.

Silent wraparound, not a crash

```
unsigned int x = 0;
--x;
std::cout << x;   // 4294967295
```

> unsigned int has no representation for -1. Subtract past zero and the result wraps to the highest value the type can hold — not garbage, the exact, defined, silent result of running out of room.

THE BUG THAT ACTUALLY BREAKS PROGRAMS

for (size_t i = n - 1; i >= 0; --i)

```
for (size_t i = n - 1; i >= 0; --i) {
    // process array[i]
}
// size_t is unsigned: when i == 0, --i wraps to a
// huge number, which is still >= 0. NEVER TERMINATES.
```

> **Why this matters** This bug produces no error, no crash, no warning. The only way to catch it is to watch i's actual value in the debugger's Watch panel — exactly what last week's debugger lecture was for.

BINARY CAN'T DO DECIMAL FRACTIONS EITHER

Just like 1/3 in decimal

Decimal

1/3 = 0.333...
forever, truncated
wherever you stop.

Binary

1/10 = 0.000110011...
forever, truncated
to 64 bits.

> 0.1 in binary is a repeating fraction, just like 1/3 is in decimal. What's actually stored is the closest 64-bit approximation — not 0.1 itself. Add two approximations, and the tiny errors don't necessarily cancel.

THE ANSWER — AND THE FIX

Compare with a tolerance, not ==

```
#include <cmath>

double a = 0.1 + 0.2, b = 0.3;
const double EPSILON = 1e-9;
if (std::abs(a - b) < EPSILON) {
    std::cout << "close enough";
}
```

The answer == on doubles asks ‘are these two bit patterns identical,’ which is almost never the question you meant to ask. epsilon is the fix, and new vocabulary you'll see again.

'A' is one character; "A" is text

```
char grade = 'A';              // one character, single quotes
std::string name = "Ada";      // text, double quotes
```

> **Key idea** Single quotes and double quotes are different types, not a style choice — a char is exactly one character; a string is text, even one character long.

ESCAPE SEQUENCES

A backslash stands in for a character you can't type

```
std::cout << "Line one\nLine two\n";   // \n -- newline
std::cout << "She said \"hi\"\n";     // \" -- a double-quote
std::cout << "C:\\Users\\ada\n";        // \\ -- a backslash
std::cout << "Tab:\there\n";            // \t -- a tab
std::cout << "\033[31mRed\033[0m";      // ANSI color, preview only
```

Key idea The same escapes work inside a char literal too — '\n' is the newline character, one char, not two. (The ANSI color escape above is an unrelated preview — a terminal convention, not a C++ feature.)

true/false, plus the ASCII behind every char

```
bool passed = true;
std::cout << passed;   // 1 -- bool prints as 1/0
std::cout << 'A' + 0;  // 65 -- adding forces promotion to int
std::cout << 'a' + 0;  // 97
std::cout << '0' + 0;  // 48 -- the CHARACTER '0' is not the NUMBER 0
```

Key idea A char is really a small integer under a text encoding (ASCII: 128 codes). Extended 8-bit encodings are sometimes loosely called ‘ANSI’ — unrelated to the ANSI escape codes for terminal color.

IMPLICIT CONVERSIONS

5 / 2 is not 2.5

```
std::cout << 5 / 2;       // 2   -- integer division truncates
std::cout << 5 / 2.0;     // 2.5 -- one operand is double
int total = 5 / 2.0;      // total is 2 -- fraction silently dropped
```

> **Discuss** Given int a = 7, b = 2; what does (a + b) / 2 print, and why?

MIXING TYPES IN ONE PROGRAM

int, double, and char in the same expressions

```
int d1 = 5, d2 = 20;
double x1 = 12.5, x2 = -5.0;
char ch1('*'), ch2('A');

std::cout << d1 + d2;   // 25 -- int + int stays int
std::cout << x1 - x2;   // 17.5 -- double + double stays double
```

Key idea The general rule (the ‘usual arithmetic conversions’): when an operator's two operands have different types, the lower-ranked one converts up to match the higher one before the operation runs — never the other way.

TYPE CONVERSION AND OUTPUT APPEARANCE

The same mechanism, printed two different ways

```
char ch1('*'), ch2('A');
std::cout << ch1 << "  " << ch2 + 3;   // *  68, not *  D
```

Key idea cout << ch1 prints the CHARACTER *. ch2 + 3 promotes ch2 to int first ('A' is 65), so it prints the NUMBER 68. Same mechanism as char/short -> int, int meeting double -> double — promotion always adds precision, never loses it.

EXPLICIT CASTING

static_cast and the older C-style cast

```
int a = 7, b = 2;
double result = static_cast<double>(a) / b;   // 3.5
double result2 = (double)a / b;                // same -- C-style cast
```

Key idea static_cast<double>(a) converts BEFORE the divide runs. It's this course's default — checked at compile time, and it says exactly what you intend.

RECAP

What can quietly go wrong with the values themselves

- Comparison operators produce a bool; && and || short-circuit — the right side is skipped entirely once the left side decides the answer.
- Char/string literals are different types; a char is really a small integer under a text encoding like ASCII.
- Overflow wraps silently — no crash, no warning — which is exactly what makes it dangerous.
- 0.1 + 0.2 == 0.3 is false everywhere, forever, by design — compare doubles with epsilon instead.
- Types promote automatically in mixed expressions; static_cast forces a conversion on purpose.

Next lecture

Control flow, and talking to the terminal

You now know what values can do wrong. Next: the tools to make decisions and repeat actions.